

Particle Filter (Lab 3)

Pravin Oli (u1999699), and Gebrecherkos Gebreslassie (u1999542)

November 19, 2024

Abstract

This report presents the implementation and evaluation of Monte Carlo Localization (MCL) using a particle filter for robot localization. The project is structured into pre-lab, lab, and post-lab activities. In the pre-lab phase, we established the theoretical foundations of MCL. During the lab session, we implemented the components of the particle filter algorithm (prediction, update, resampling). We used two resampling methods: Roulette Wheel Resampling and Stochastic Universal Sampling, comparing their effects on particle diversity. A simulated differential drive robot was used for testing, utilizing the `DifferentialDriveSimulatedRobot` class to mimic real-world robot motion and sensor data. The results demonstrated that the particle filter effectively localized the robot over time, with particles converging around the true trajectory. Stochastic Universal Sampling proved superior in maintaining particle diversity and proportional weight representation compared to Roulette Wheel Resampling. Challenges such as missing map initialization, unrealistic particle propagation due to motion noise, discrepancies between actual and simulated measurements, and resampling errors were identified and addressed.

Report Contents

1. Introduction
2. Simulation and Localization
3. Particle Filter-Based Localization
4. Result and Discussion
5. Issues and Solutions
6. Conclusion

1 Introduction

Lab Activities

- Pre-Lab Activity
- Lab Activity
- Post-Lab Activity

1.1 Pre-Lab

In the *Pre-Lab* session, we prepared for the lab by understanding the theoretical foundation of MCL, and setting up the simulation framework.

Monte Carlo Localization (MCL)

MCL uses Bayesian filtering to estimate the robot's state x_t at time t , given sensor measurements $z_{1:t}$ and control inputs $u_{1:t}$. The posterior distribution $p(x_t \mid z_{1:t}, u_{1:t})$ is computed using the Bayes filter equation:

$$p(x_t \mid z_{1:t}, u_{1:t}) \propto p(z_t \mid x_t) \int p(x_t \mid x_{t-1}, u_t) p(x_{t-1} \mid z_{1:t-1}, u_{1:t-1}) dx_{t-1}.$$

- $p(z_t \mid x_t)$: Measurement model (likelihood of observations given the state).
- $p(x_t \mid x_{t-1}, u_t)$: Motion model (state transition probability given control inputs).

Particle Representation and Algorithm Steps

The posterior is approximated using particles $\{x_t^{[i]}, w_t^{[i]}\}_{i=1}^N$, where each particle $x_t^{[i]}$ represents a possible state, and $w_t^{[i]}$ is its weight. The MCL algorithm includes:

1. **Prediction:** Particles are updated using the motion model:

$$x_t^{[i]} \sim p(x_t \mid x_{t-1}^{[i]}, u_t).$$

2. **Update:** Weights are updated based on the measurement model:

$$w_t^{[i]} = p(z_t \mid x_t^{[i]}).$$

3. **Resampling:** Particles are resampled based on their weights to avoid degeneracy.

Monte Carlo Approximation

The posterior distribution is approximated as:

$$p(x_t \mid z_{1:t}, u_{1:t}) \approx \sum_{i=1}^N w_t^{[i]} \delta(x_t - x_t^{[i]}),$$

where δ is the Dirac delta function.

1.2 Lab Session

During the lab session, we focused on implementing two critical steps of the Particle Filter algorithm:

- **Prediction (part-1):** In this part we tried to Implement the prediction step, where particles are propagated using the motion model.
 - Completed `Prediction()` in the `ParticleFilter` class.
 - Added logic to `SampleProcessModel()` in the `PF_3DOF` class.
- **Update (part-2):** Implemented the update step, where particles are weighted based on sensor data.
 - Programmed `ReadRanges()` and `GetMeasurements()`.
 - Completed the `Update()` and resampling methods (`RouletteWheelResampling()` and `StochasticUniversalResampling()`).

1.3 Post-Lab

In the *Post-Lab* session, we debugged our implementation, visualized particle evolution over time, and analyzed localization performance to finalize the report.

2 Simulation and Localization

2.1 Simulation

The class `DifferentialDriveSimulatedRobot` inherits from the `SimulatedRobot` class and overrides key methods to simulate the robot's motion, sensors, and state updates. It is used to simulate a differential drive robot's movement and sensor readings in a controlled environment, supports visualizing the robot's trajectory, and simulating noisy sensor data. It includes the following basic functions:

- **Initialization** (`__init__`): Initializes robot state (position, orientation, velocity).
- **Motion Model** (`fs`): Updates robot's state (pose and velocity) based on control inputs (linear and angular velocities).
- **Sensor Simulations:** Range Measurements (`ReadRanges`), Encoder Readings (`ReadEncoders`), Compass Readings (`ReadCompass`).
- **Visualization:** `PlotRobot`.

2.2 Localization

The class `MCLocalization` inherits from `ParticleFilter` and `Localization`. It implements the key concepts of Monte Carlo Localization (MCL), where the particles represent possible robot poses, and the algorithm updates these particles based on control inputs and sensor measurements.

- **Initialization** (`__init__`): Initializes both the `ParticleFilter` and `Localization` classes by calling their constructors (`super()`).
- **GetMeasurements()**: This function is designed to get sensor measurements, implemented in a child class that extends `MCLocalization`.
- **Localize()**: The `Localize()` method is responsible for performing a single localization iteration.
- **LocalizationLoop()**: Calls `Localize()` for each step to compute the robot's pose.
- **Plotting**.

3 Particle Filter-Based Localization

3.1 Prediction

- **Motion Model**: Describes how particles move from one state to another, given the control inputs.
- **Control Input** (`u`): Represents the actions or movements that the system can take (e.g., moving forward or turning).
- **Covariance** (`Q`): Represents the uncertainty or noise in the control input, which affects the particles' predictions.

Code Snippet:

```
def Prediction(self, u, Q):
    for i in range(len(self.particles)):
        self.particles[i] = self.SampleProcessModel(self.particles[i], u, Q)
```

3.2 Update

- **Weight Update**: Each particle's weight is updated based on how well its predicted measurement matches the actual sensor measurement.
- **Normalization**: After updating the weights, they are normalized to sum to 1.
- **Effective Number of Particles** (`n_eff`): A measure of how many particles are effectively contributing to the estimation. Resampling occurs if `n_eff` is below a threshold.

Code Snippet:

```
def Update(self, z, R):
    for i, particle in enumerate(self.particles):
        simulated_measurement = self.ObservationModel(particle, z, R)
        self.particle_weights[i] = self.pdf(simulated_measurement, R, z)

    self.particle_weights /= np.sum(self.particle_weights)

    n_eff = 1.0 / np.sum(self.particle_weights**2)
    if n_eff < self.resampling_threshold:
        self.resampling_function()
```

3.3 Resampling

We used two methods of resampling: Roulette Wheel Resampling and Stochastic Universal Resampling. Below is a comparison:

Feature	Roulette Wheel Resampling	Stochastic Universal Sampling
Randomness	High randomness	Lower randomness
Particle Diversity	Poor diversity	Better diversity
Fairness in Weight Usage	Uneven	Proportional representation
Computational Complexity	$O(N \log N)$	$O(N)$

Table 1: Comparison of Resampling Methods

Code Snippet:

```
def RouletteWheelResampling(self):
    cumulative_sum = np.cumsum(self.particle_weights)
    cumulative_sum[-1] = 1.0
    new_particles = []

    for _ in range(len(self.particles)):
        pick = np.random.rand()
        index = np.searchsorted(cumulative_sum, pick)
        new_particles.append(self.particles[index])
    self.particles = new_particles
```

4 Results and Discussion

Choosing the sampling method is user input by selecting 1 or 2. Figure 1 shows the output interface, Hence we performed the initialization of particles in a simulated robot localization system, with 100 particles scattered across the environment.

The distribution reflects the starting uncertainty in the robot’s position. The Figure-2 illustrates the indices of particles selected using Roulette Wheel Resampling, demonstrating the probabilistic nature of the technique, where higher-weighted particles are more frequently chosen, though with potential loss of diversity.

The Figure-3 presents indices from Stochastic Universal Sampling (SUS), which ensures a more balanced particle selection, maintaining diversity and proportional representation.

The Figure-4 represents the initial state of the particle filter, where particles are widely scattered, indicating low accuracy in localization.

The Figure-5 shows the refined particle distribution after iterative updates, where particles converge closely around the robot’s true trajectory, demonstrating the improved accuracy and effectiveness of the particle filter in estimating the robot’s position.

5 Issues and Solutions

- **Problem:** Missing map initialization and incorrect particle sampling.
- **Solution:** Ensured `map_landmarks` was initialized and particles were sampled using Pose3D objects.
- **Problem:** Noise in motion caused unrealistic particle propagation.
- **Solution:** Adjusted noise covariance matrices ($\mathbf{Q}_k$) for better realism.
- **Problem:** Discrepancy between actual and simulated range measurements.
- **Solution:** Refined Observation Model and verified covariance consistency in the probability density function (pdf).
- **Problem:** Stochastic resampling occasionally failed due to weight normalization errors.
- **Solution:** Debugged weight normalization and ensured resampling thresholds were dynamically checked.

6 Conclusion

In this lab, we successfully implemented and tested the Monte Carlo Localization (MCL) algorithm using a particle filter framework. The lab activities were divided into pre-lab, lab, and post-lab phases, each addressing critical components of the MCL process. The iterative process of prediction, update, and resampling effectively localized the robot, with Stochastic Universal Sampling outperforming Roulette Wheel Resampling in maintaining particle diversity. Key challenges, including noise in motion models and resampling errors, were addressed to improve accuracy. Results demonstrated the system’s ability to refine particle distributions and accurately estimate the robot’s trajectory.

